End-to-End Testing with CodeceptJS

Your Storefront Next app includes an end-to-end test suite built with CodeceptJS and Playwright. Run automated tests against your storefront locally or in a remote environment to verify that user-facing flows work as expected.

Contents

Before You Begin

Configure the storefront app from the project root.

1. Copy `.env.default`, the included environment template, or start from any `.env` file you already have configured.

```
1  cp .env.default .env
```

2. Edit `.env` with your B2C Commerce credentials.

Set Up Your Environment

From the project root, install dependencies.

```
1  pnpm install
```

This step also installs Playwright browsers.

Configure the Test Environment

```
cp e2e/.env.sample e2e/.env
```

This table describes the key variables in `.env`.

Variable	Default
`BASE_URL`	`http://localhost:517`
`SITE_ID`	`RefArchGlobal`
`SITE_ALIAS`	global
`LOCALE`	en-GB
`HEADLESS`	`false`
`VERBOSE`	`false`

Run Your First Test

```
# Run all tests against a runni
pnpm e2e

# Auto-start local dev server a
pnpm e2e --mode=local
```

Run Tests

Run all commands from the template root.

`pnpm e2e --mode=local`	Auto-start local dev server and run tests
`pnpm e2e --mode=remote`	Run against a remote URL (requires `BASE_URL`)
`pnpm e2e --grep "@checkout"`	Filter tests by tag or name
`pnpm e2e --headed`	Run tests with the browser visible
`pnpm e2e --ui`	Open interactive UI mode
`pnpm e2e --debug`	Debug with the CodeceptJS inspector
`pnpm report`	Open the Allure test report

Use AI Features

The test suite integrates with CodeceptJS AI to provide self-healing tests, an interactive debugging console, and automatic page object generation. AI features are disabled by default. To use them, pass the `--ai` flag.

Set Up AI Features

1. Get an API key from console.anthropic.com.
2. Add the key to `.env`.

```
ANTHROPIC_API_KEY=sk-ant-..
```

3. Run tests with AI enabled.

```
pnpm e2e --ai
```

AI Capabilities

plain English.

- **Page object generation**: Call `I.askForPageObject()` to read the live document object model (DOM) and generate a page object.

To see AI healing decisions in the console, run:

```
DEBUG="codeceptjs:ai" pnpm e2e
```

Generate Tests with AI Coding Assistants

AI coding assistants such as Cursor and Claude Code can generate complete E2E tests using the `generate-storefront-e2e-test` skill. This is separate from the Anthropic API key used for runtime self-healing–no additional API key is required.

The skill guides the assistant through a structured workflow, from understanding the commerce scenario to producing spec files, page objects, and self-healing recipes.

Use the Skill

Ask your AI assistant to create an E2E test. Any of these prompts activates the skill.

```
"Create an E2E test for the che
"Write a test that validates pr
"Add E2E coverage for the shopp
```

The assistant:

1. **Clarifies requirements**–asks about scope, device targets, and guest or authenticated flows
2. **Audits existing coverage**–reviews unit tests and Storybook stories to avoid duplication
3. **Proposes a test plan**–lists scenarios, tags, and page objects before writing code

`src/pages/index.ts`, `src/flows/index.ts`, and `helpers/self-healing/recipes.ts`

Generated Artifacts

This table describes the files the skill produces.

Artifact	Location
Spec file	`src/specs/<feature>/*.`
Page object	`src/pages/*.page.ts`
Flow	`src/flows/*.flow.ts`
Healing recipes	`helpers/self-healing/recipes.ts`

Conventions

Generated code follows these conventions.

- Scenarios never call `I.*` directly–all browser interactions live in page objects or flows.
- Chai `expect()` for value assertions, CodeceptJS methods for UI interactions.
- All hardcoded paths wrapped in `buildSitePath()` for multisite support.
- Each scenario is independent and can run in any order.
- Page objects use semantic locators with `.as('Name')` descriptions.

After generation, verify the tests.

```
pnpm e2e --grep "@your-tag"
```

before each test run. To generate them manually for IDE IntelliSense, run:

```
pnpm def
```

Accessibility Testing

The end-to-end test suite includes full-page accessibility scanning using axe-core via `@axe-core/playwright`. Scans run in a real browser against the running storefront and catch issues that component-level Storybook a11y tests can't detect, such as page composition, routing, layout, and responsive behavior issues.

How It Works

- **Scope**: WCAG 2.1 AA (`wcag2a`, `wcag2aa`, `wcag21aa` tags)
- **Viewports**: Desktop (1200×900) and mobile (Pixel 7 emulation). `pnpm a11y` runs both passes sequentially via `scripts/run-a11y.ts`, starting with desktop and then enabling mobile with `PLAYWRIGHT_MOBILE=true`.
- **Baseline**: The test suite snapshots violation counts per page per viewport. CI fails only when critical or serious violations increase or a new critical/serious rule appears. Moderate and minor regressions are logged as informational and don't block CI.
- **Retries**: The test suite retries infrastructure failures such as timeouts and navigation errors up to two times via `.retry(2)` on the Feature. The `a11yNoRetry` plugin never retries confirmed a11y regressions (thrown as `A11yBaselineError`) because axe scans are deterministic and re-running a real violation always reproduces it.

development server before running scans. From `packages/template-retail-rsc-app`, run `pnpm dev`.

```
# Run a11y scans against the ru
pnpm a11y

# Generate a Markdown report wi
pnpm a11y:report

# Scan all pages and write exis
# Typically useful for gating n
pnpm a11y:update-baseline
```

Console Output

Each scan prints a banner identifying the page, viewport, and WCAG standard.

```
═══════════════════════════════
  A11Y SCAN: homepage | desktop
  Standard: WCAG 2.1 AA (wcag2a
═══════════════════════════════
```

The test suite prints the severity legend once at the start of the run. On completion, each scan prints one of these results.

- `✓ PASS: homepage/desktop—5 violations (within baseline)` –scan passed
- `↓ homepage/desktop— violations decreased, run pnpm a11y:update-baseline` – improvement detected
- `ⓘ 2 moderate/minor rules exceeded baseline (not blocking—update with pnpm a11y:update-baseline)` – moderate or minor counts increased; informational only, printed below the PASS line
- Failure: full violation report followed by a summary of new or increased critical/serious rules

result collection enabled and writes two files to `a11y-report/`.

- **`report.md`** –Markdown summary table (page × viewport × severity counts) and per-violation detail with rule ID, description, axe-core help URL, and HTML snippets of affected elements. Use this to create accessibility tickets.
- **`report.html`** –Styled HTML version of the same report, intended for CI artifact upload and browser viewing.

`a11y-report/` is gitignored and treated as a generated artifact. Generate it locally when needed.

```
pnpm a11y:report
# Writes a11y-report/report.md
```

How the Baseline Works

The test suite stores the baseline in `a11y-baseline.json` at the package root. Each entry maps a `<pageKey>/<viewport>` key to a `Record<ruleId, nodeCount>`.

```
{
  "homepage/desktop": { "color-
  "homepage/mobile": { "color-c
  "login/desktop": {}
}
```

On every test run, the spec compares the current axe output against the baseline.

- **New critical/serious rule** or **more critical/serious violations** than baseline → the test fails
- **New moderate/minor rule** or **more moderate/minor violations** than baseline → the test passes and logs the violation
- **Fewer violations** than baseline → the test passes with a log message suggesting a baseline update

Ratchet-down workflow: After fixing a known issue, update the baseline to lock in the improvement.

```
4  git commit -m "chore: ratchet a
```

Initial setup for new pages: After adding a new page, run `pnpm a11y:update-baseline`. The command creates baseline entries automatically and populates them with current violation counts. Commit the result.

Run at a Specific Viewport

`pnpm a11y` always runs both viewport passes. To target a single viewport, invoke the test runner directly.

```
1  # Desktop only
2  tsx src/scripts/cli/test-runner
3
4  # Mobile only (Pixel 7 emulatio
5  PLAYWRIGHT_MOBILE=true tsx src/
```

> **Note**
>
> `pnpm e2e run --multiple desktop --grep "@a11y"` doesn't work. CodeceptJS `run-multiple` doesn't forward `--grep` to its workers, so all tests run instead of just the a11y suite.

Add a New Page

1. Add a `Scenario` to `src/specs/core/accessibility.spec.ts`. Use this pattern as a starting point.

   ```
   1  Scenario("My New Page acces
   2    myPage.navigate();
   3    await scanAndAssert("my-
   4  })
   5    .tag("@a11y")
   6    .tag("@my-new-page");
   ```

2. Run `pnpm a11y:update-baseline`. The command creates baseline entries for the new page automatically.
3. Review the diff in `a11y-baseline.json`, then commit it.

Level	Meaning
`critical`	Blocks access entirely for users with disabilities
`serious`	Creates significant barriers; fix quickly
`moderate`	Creates difficulty for users with disabilities. Workarounds sometimes exist.
`minor`	Low impact; fix when convenient

Severity appears in these places in test output.

- **Scan banner**: Printed before each scan alongside the WCAG standard.
- **Failure summary**: Each failing rule appears inline as `rule-id [impact]: N violations` so you can assess priority at a glance.
- **Full violation report**: Printed above the summary, grouped by impact level with rule descriptions, help URLs, and affected element selectors and HTML snippets.
- **Markdown report**: Summary table and per-rule detail sections with help URLs and HTML snippets for ticket creation.

Environment Variables

This table describes the environment variables for the a11y suite.

Variable	Values	D
`BASE_URL`	URL	St D `h`
`PLAYWRIGHT_MOBILE`	`true`	Er `p` au m to
`A11Y_UPDATE_BASELINE`	`true`	W co ba

		se
`A11Y_COLLECT_RESULTS`	`true`	W a of p th

CI Integration

A11y scans run as a parallel job alongside the core E2E suite on every PR, merge group event, and push to `main` or `release-*` branches. Both jobs share the same MRT deployment, so there's no extra deploy cost.

- **Caller workflow**: `.github/workflows/e2e-core-pr.yml`
- **Reusable runner**: `.github/workflows/e2e-pr-runner.yml` –the `run_a11y` job handles the scan, reporting, and artifact upload.
- **Kill switch**: The `run_a11y` job is gated on the `ENABLE_A11Y_CI` repository variable. Set it to `false` to disable a11y CI globally without modifying any workflow file.
- **Command**: `pnpm a11y:report --no-ai` –runs scans in collect mode and generates both `a11y-report/report.md` and `a11y-report/report.html`.
- **Job summary**: The CI job appends `a11y-report/report.md` to the GitHub Actions job summary after each run, so you can view violation details directly in the Actions UI without downloading anything.
- **HTML report artifact**: The CI job uploads `a11y-report/report.html` as a CI artifact (`a11y-report-<run-id>`) after each run. Download it from the Actions run summary to view violation details in a browser without cloning the branch.

Troubleshooting

TypeScript errors about missing page objects

Tests fail with "Element not found"

Enable AI self-healing.

```
pnpm e2e --ai --grep "@failing-
```

Local dev server won't start

Check that `.env` exists at the project root and contains valid B2C Commerce credentials.

Port 5173 is already in use

```
lsof -ti:5173 | xargs kill
```

AI features aren't working

Verify that `ANTHROPIC_API_KEY` is set in `.env`.

```
cat .env | grep ANTHROPIC
```

Remote tests fail: "BASE_URL required"

Set `BASE_URL` before running the command.

```
BASE_URL=https://your-site.com
```

DID THIS ARTICLE SOLVE YOUR ISSUE?
Let us know so we can improve!

Share your feedback

Try for free

Commerce Cloud
Lightning Design System
Einstein
Quip

Trailhead
Sample Apps
AppExchange

Blog
Salesforce
Salesforce